DevMentor AI - By Dhruv Shukla

06 Aug 2026

Daily Learning Report

TODAY'S LEARNING

Operators

Day 2 | Beginner | 10 min read

Generated by DevMentor AI by Dhruv Shukla

🎯 Today's Goal

By the end of this module, Dhruv will be able to construct, evaluate, and optimize expression logic in C# using arithmetic, relational, logical, null-coalescing, and bitwise operators. He will confidently prevent common runtime bugs like integer overflow, null reference exceptions, and truncation errors in production applications.

📘 Core Concept

Operators are specialized symbols that instruct the C# compiler to perform mathematical, logical, or value manipulation operations on one or more operands. They form the foundational engine for all data processing and conditional execution in .NET programs.

How It Works Internally

At compile time, C# translates operators into standard Intermediate Language (IL) instructions. For example, + becomes add, and == becomes ceq (compare equal).

- Precedence & Associativity: Precedence dictates which operator runs first in an expression (e.g., * before +). Associativity determines evaluation direction (left-to-right or right-to-left) when operators share the same precedence.
- Short-Circuit Evaluation: Logical operators && (AND) and || (OR) evaluate left-to-right and stop immediately if the ultimate boolean result is guaranteed. If the left side of && is false, the right side is never evaluated.

Key Rules & Categories

*Arithmetic Operators (+, -, , /, %):** Standard mathematical operations. Integer division (int / int) truncates decimals.

Relational Operators (==, !=, <, >, <=, >=): Compare two values and return a bool.

Logical Operators (&&, ||, !): Evaluate boolean expressions using short-circuit logic.

Null-Coalescing Operators (??, ??=): Provide fallback values for nullable types or assign a value only if the operand is null.

Checked/Unchecked Context: By default, arithmetic integer overflow wraps around silently in C# unless wrapped in a checked block.

What Happens If You Do It Wrong

Using single & or | instead of && or || disables short-circuiting, leading to unexpected NullReferenceException crashes when checking nulls. Neglecting integer division rules results in silent precision loss in financial logic.

```
using System;

namespace OperatorBasics
{
    public class Program
    {
        public static void Main()
        {
            // Null-Coalescing Assignment (??=)
            string? username = null;
            username ??= "GuestUser";

            // Short-circuiting logic preventing NullReferenceException
            string? payload = null;
            bool isValid = payload != null && payload.Length > 0;

            // Integer Division Truncation vs Double Division
            int intResult = 5 / 2;         // Evaluates to 2
            double doubleResult = 5.0 / 2; // Evaluates to 2.5

            Console.WriteLine($"User: {username}, IsValid: {isValid}");
            Console.WriteLine($"Int Div: {intResult}, Double Div: {doubleResult}");
        }
    }
}
```

💼 Real Project Example

In an e-commerce order processing service, calculating line items requires dynamic pricing rules, tax additions, and default parameter fallback handling without runtime failures.

```
using System;
```

```
namespace EcommerceApp.Services
{
    public record OrderRequest(decimal BasePrice, decimal? DiscountAmount, int Quantity, bool
IsTaxExempt);

    public interface IPricingService
    {
        decimal CalculateTotalPrice(OrderRequest request);
    }

    public class PricingService : IPricingService
    {
        private const decimal DefaultTaxRate = 0.10m; // 10% tax rate

        public decimal CalculateTotalPrice(OrderRequest request)
        {
            if (request == null || request.Quantity <= 0)
            {
                throw new ArgumentException("Invalid order parameters.");
            }

            // Null-coalescing operator ensures discount defaults to 0 if null
            decimal discount = request.DiscountAmount ?? 0.0m;

            // Arithmetic & Ternary Operators
            decimal discountedUnitPrice = request.BasePrice - discount;
            decimal subtotal = discountedUnitPrice * request.Quantity;

            // Short-circuit evaluation for tax eligibility
            decimal taxRate = request.IsTaxExempt || subtotal <= 0 ? 0.0m : DefaultTaxRate;

            decimal totalTax = subtotal * taxRate;
            return subtotal + totalTax;
        }
    }
}
```

Explanation & Senior Engineer Advice

- Null Fallbacks (??): Safely unwraps DiscountAmount without verbose if-else blocks.
- Ternary Operator (? :): Conditionally assigns taxRate cleanly.
- Senior Tip: Prefer decimal over double for financial calculations to prevent floating-point rounding inaccuracies inherent to binary representations.

⚠️ Top 3 Mistakes

1. Integer Division Truncation Loss

Developers perform calculations expecting fractional results, but integer operands discard the fractional part completely.

❌ Bad Code:

```
int totalItems = 50;
int completedItems = 12;
double percentage = (completedItems / totalItems) * 100; // Evaluates to 0!
```

Why it fails: 12 / 50 executes integer division, yielding 0 before it ever casts or multiplies by 100.

Double Check Fix:

```
int totalItems = 50;
int completedItems = 12;
double percentage = ((double)completedItems / totalItems) * 100; // Evaluates to 24.0
```

---

2. Using Bitwise Operators Instead of Short-Circuit Operators

Using & instead of && forces both sides of the condition to evaluate, ignoring safe null checks.

❌ Bad Code:

```
string? input = null;
if (input != null & input.Length > 0) // Throws NullReferenceException!
{
    // Do work
}
```

Why it fails: & is a logical non-short-circuiting operator in this context. It evaluates input.Length even when input is null.

Double Check Fix:

```
string? input = null;
if (input != null && input.Length > 0) // Evaluates safely to false
{
    // Do work
}
```

---

3. Misunderstanding Operator Precedence in Mixed Expressions

Combining ternary operators with addition or string concatenation without parentheses leads to incorrect execution order.

❌ Bad Code:

```
bool isMember = true;
string message = "Total cost: " + isMember ? "$10" : "$20"; // Compiler Error or Bug
```

Why it fails: The string concatenation + has higher precedence than ? :, evaluating "Total cost: " + isMember first, which breaks the dynamic condition.

Double Check Fix:

```
bool isMember = true;
string message = "Total cost: " + (isMember ? "$10" : "$20");
```

Industry News

- Test reporting in Microsoft.Testing.Platform: from red build to root cause

Microsoft introduces enhanced test reporting capability directly into the Microsoft.Testing.Platform. This update bridges build failures to actionable root-cause reports in .NET environments. Understanding robust diagnostics helps software architects write reliable test suites that validate code health across enterprise pipelines.

🔗 Read full article

- How we took malware advisories beyond npm

GitHub Security expanded its supply-chain malware advisories beyond npm to cover multiple language ecosystems. This effort proactively scans, detects, and isolates malicious packages across open-source ecosystems. Developing secure architecture requires strict reliance on audited dependencies and defensive coding practices.

🔗 Read full article

- From Projects to Products: Turning Platforms into Products People Use

InfoQ covers the transition in platform engineering from short-term project deliverables to long-term internal platform products. This strategy focuses on internal developer experience, scalability, and lifecycle stability. For

a modern architect, treating APIs and internal frameworks as products improves team velocity.

Read full article

- Presentation: From ms to µs: OSS Valkey Architecture Patterns for Modern AI

This architectural review demonstrates how Valkey (an open-source Redis fork) achieves microsecond latency for AI memory state stores. High-performance computing relies heavily on optimized memory access patterns and low-level data structure manipulation. Engineers learn how raw performance choices impact large-scale system throughput.

Read full article

- Wiz Discloses CosmosEscape, and Practitioners Debate What Customers Could Have Done

Wiz discovered a critical security vulnerability involving Azure Cosmos DB key handling, raising architecture security discussions. The vulnerability emphasizes the importance of defensive identity management over shared master key access. Designing cloud systems requires strict least-privilege principles to mitigate infrastructure access risks.

Read full article

- Article: Runtime-Agnostic AI Workflows: A Pattern for Production Durability and Fast Eval Iteration

This article details decoupling AI orchestration logic from underlying cloud runtime dependencies to ensure system durability. Creating runtime-agnostic designs allows teams to iterate quickly without rewriting core business rules. It highlights abstraction principles essential for long-term software architecture.

Read full article

- Pods as Workers, Not Agents: Rethinking the Deployment Unit for AI Agents on Kubernetes

InfoQ discusses architectural patterns for running AI workloads as bounded, predictable Kubernetes worker pods rather than autonomous agents. This paradigm shift provides higher reliability, clear state management, and better resource allocation in distributed environments. Understanding deployment boundaries is vital for cloud-native architects.

Read full article

❓ Interview Questions & Answers

Q1: What is the difference between == and .Equals() in C#?

A1: The == operator compares reference identity for reference types by default (unless overloaded) and value equality for value types. .Equals() is a virtual method that compares object contents for value equality across both value and reference types when overridden.

```
string a = new string(new char[] {'h', 'e'});
string b = new string(new char[] {'h', 'e'});
bool opResult = (a == b);       // True (String overloads == for value equality)
bool eqResult = a.Equals(b);   // True
```

Q2: How does short-circuiting work with logical operators (&& and ||)?

A2: Short-circuiting evaluates compound expressions from left to right and stops evaluation as soon as the final result is known. For &&, if the left condition is false, the right condition is skipped. For ||, if the left condition is true, the right condition is skipped.

```
User? user = null;
// Safe: user.IsActive is never called because user != null evaluates to false
if (user != null && user.IsActive) { }
```

Q3: What are the null-coalescing (??) and null-coalescing assignment (??=) operators?

A3: The ?? operator returns the left operand if it is not null; otherwise, it returns the right operand. The ??= operator assigns the right operand to the left operand only if the left operand evaluates to null.

```
List<string>? items = null;
items ??= new List<string>(); // Instantiates list only because items was null
string name = null;
string displayName = name ?? "Anonymous"; // Fallback to "Anonymous"
```

Q4: What is the difference between prefix (++x) and postfix (x++) increment operators?

A4: The prefix increment (++x) increments the variable value first and then returns the newly updated result to the expression. The postfix increment (x++) evaluates the expression using the current variable value first, and increments the variable afterward.

```
int a = 5;
int b = ++a; // a becomes 6, b receives 6

int x = 5;
int y = x++; // y receives 5, x becomes 6
```

Q5: What is the purpose of checked and unchecked context operators in C#?

A5: By default, arithmetic operations execute in an unchecked context where integer overflows wrap around silently without errors. The checked operator forces the runtime to throw an OverflowException if an operation exceeds the maximum value of the data type.

```
int max = int.MaxValue;
// int overflow = max + 1; // Unchecked: yields -2147483648
int safe = checked(max + 1); // Throws System.OverflowException
```

Q6: What is operator overloading in C#, and what are its key restrictions?

A6: Operator overloading allows custom types (classes/structs) to define custom behavior for built-in C# operators using static methods with the operator keyword. Key rules require that overloaded operators must be public static methods, and complementary operators (like == and !=) must be overloaded in pairs.

```
public readonly struct Money
{
    public decimal Amount { get; }
    public Money(decimal amount) => Amount = amount;

    public static Money operator +(Money a, Money b)
        => new Money(a.Amount + b.Amount);
}
```

📚 Revision Summary

Revision Topic: Day 1 — Variables and Data Types

- Key Idea: Variables are named memory locations holding typed data. C# enforces strong typing using Value Types (stored on the stack, containing raw values) and Reference Types (stored on the heap, containing memory addresses pointing to data).
- Main Thing to Remember: Value types (int, struct, bool) are copied by value, whereas Reference Types (string, class, object) copy reference addresses, meaning multiple variables can point to the exact same heap memory object.

🚀 Tomorrow Preview

Tomorrow we will cover Control Flow (Conditional Statements and Switch Expressions).

You will build directly on today's operator mechanics to execute execution pathways using modern C# pattern matching and concise decision structures.